



UNIVERSIDAD POLITÉCNICA DE PACHUCA

INGENIERÍA EN SOFTWARE

MINERÍA DE DATOS

Grupo: 09_02

PROYECTO INTEGRADOR SESIÓN 14

Nombre del Alumno:

Alvaro Lechuga Vecino

Teodoro Gómez Santillan

Alan Khalid Licona Ramírez

Glosario de Términos Técnicos

Término	Definición
AppTest	Framework de pruebas automatizadas de Streamlit que ejecuta un script completo en un entorno controlado y permite inspeccionar excepciones, métricas y advertencias sin abrir un navegador.
Caché (<code>st.cache_data</code> / <code>st.cache_resource</code>)	Mecanismo de Streamlit que evita recalcular o recargar datos/modelos costosos (como el modelo de embeddings) en cada interacción del usuario con la interfaz.
Dashboard	Interfaz visual interactiva que consolida múltiples fuentes de datos y resultados en una sola vista navegable, en este caso mediante pestañas temáticas.
Dice / Slice	Operaciones OLAP: Slice filtra por un único valor de una dimensión (ej. una selección); Dice filtra por la combinación de varios valores de distintas dimensiones a la vez.
Embeddings semánticos	Representación numérica de un texto que captura su significado, permitiendo comparar similitud de contenido aunque no compartan las mismas palabras exactas.
Heatmap	Visualización tipo matriz donde el color de cada celda representa la magnitud de un valor; se usó para mostrar la operación Pivot del cubo OLAP (Liga × Categoría de posición).
Pivot	Operación OLAP que intercambia el orden de las dimensiones mostradas en una tabla o visualización, sin cambiar el contenido analítico subyacente.
Roll-up	Operación OLAP que agrega datos subiendo a un nivel más general dentro de una jerarquía (ej. de Selección a Grupo).
Streamlit	Framework de Python para construir aplicaciones web interactivas de análisis de datos con relativamente poco código, ejecutado localmente en este proyecto.
Trendline (OLS)	Línea de tendencia ajustada por mínimos cuadrados ordinarios que se superpone a un diagrama de dispersión para visualizar la relación lineal entre dos variables.

Objetivo de la Sesión

Construir un dashboard interactivo, ejecutado localmente con Streamlit, que consolide en una sola interfaz navegable los hallazgos de las 5 semanas del proyecto: el pipeline de minería de texto (corpus, clusters temáticos, pronóstico, anomalías y buscador BM25/semántico), el cubo OLAP real construido en la Sesión 13 sobre datos de jugadores y equipos, y la trazabilidad metodológica BRAI del proyecto completo. Este dashboard cumple el rol de "Producto 1: Sistema funcional" exigido por el curso, y sirve como apoyo visual directo para la defensa técnica de la Sesión 15.

Herramientas y Tecnologías Utilizadas

- Streamlit — framework de construcción de la aplicación web interactiva, ejecutada localmente.
- pandas, plotly — manipulación de datos y visualizaciones interactivas (barras, heatmap, dispersión).
- rank_bm25 — construcción del índice léxico para el buscador interactivo.
- sentence-transformers (paraphrase-multilingual-MiniLM-L12-v2) — modelo de embeddings para la búsqueda semántica en vivo.
- statsmodels — línea de tendencia (OLS) en el diagrama de dispersión de la Pregunta 1.
- Streamlit AppTest — framework de pruebas automatizadas usado para validar el script antes de cada entrega, sin necesidad de abrir un navegador.

Roadmap de la Sesión

- 1. Definición de 3 pestañas temáticas: Minería de Texto, Rendimiento Real y Metodología BRAI.
- 2. Construcción de la primera versión de la aplicación, con carga de datos cacheada.
- 3. Validación automatizada en un entorno de pruebas, detectando y corrigiendo una dependencia faltante antes de la primera entrega.
- 4. Primera ejecución real por el usuario: error real (AttributeError) por una estructura de datos JSON distinta a la asumida.
- 5. Diagnóstico dirigido: solicitud de la estructura exacta de los 3 archivos JSON involucrados, en vez de seguir adivinando.
- 6. Reescritura de la sección de minería de texto conforme a la estructura real confirmada.
- 7. Segunda validación automatizada, replicando fielmente la estructura real de los datos del proyecto.
- 8. Corrección de un segundo error de uso (comando de ejecución incorrecto) reportado por el usuario.
- 9. Validación visual completa de las 3 pestañas, con capturas de pantalla reales tomadas en la máquina del usuario.

Evidencia visual del sistema funcionando

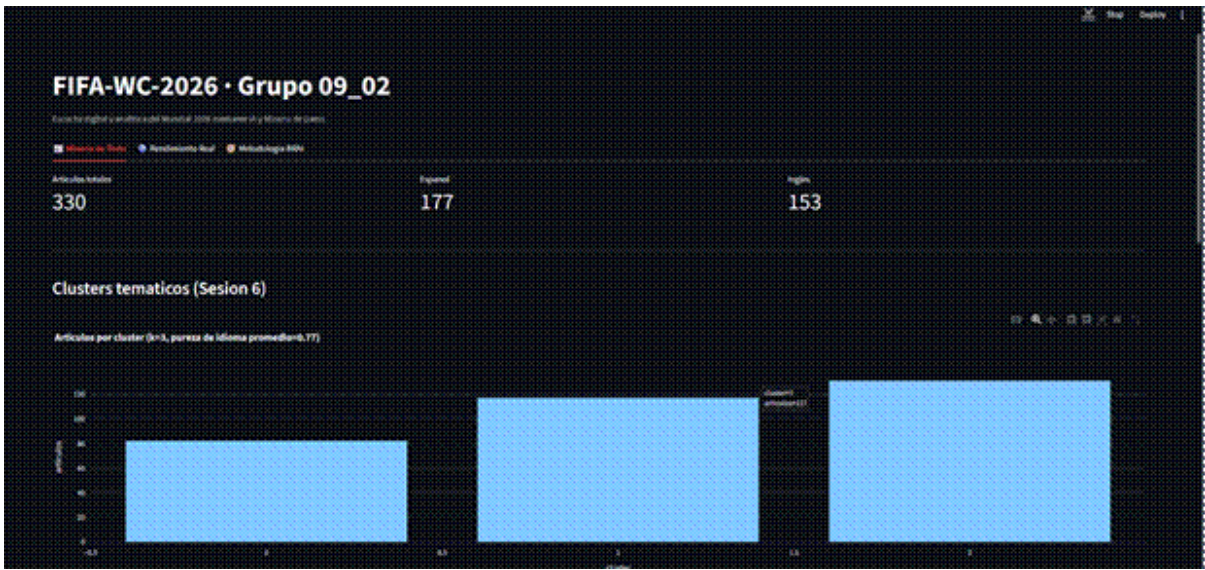


Figura 1. Pestaña Minería de Texto — resumen del corpus (330 artículos) y clusters temáticos (k=3).

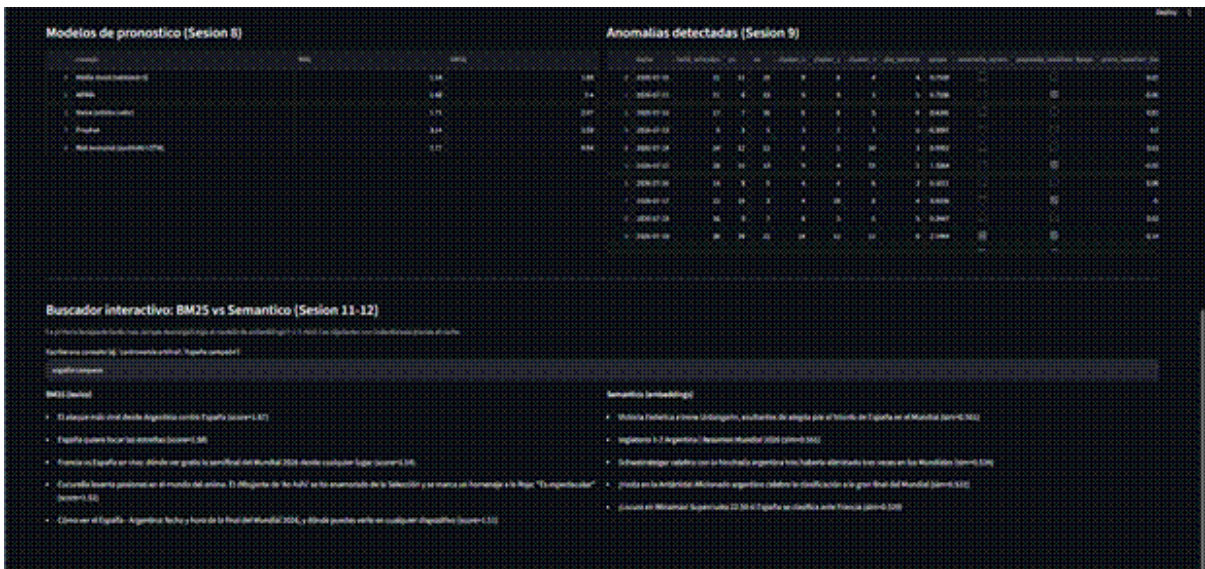


Figura 2. Comparación de modelos de pronóstico, anomalías detectadas, y buscador BM25 vs. semántico en vivo.

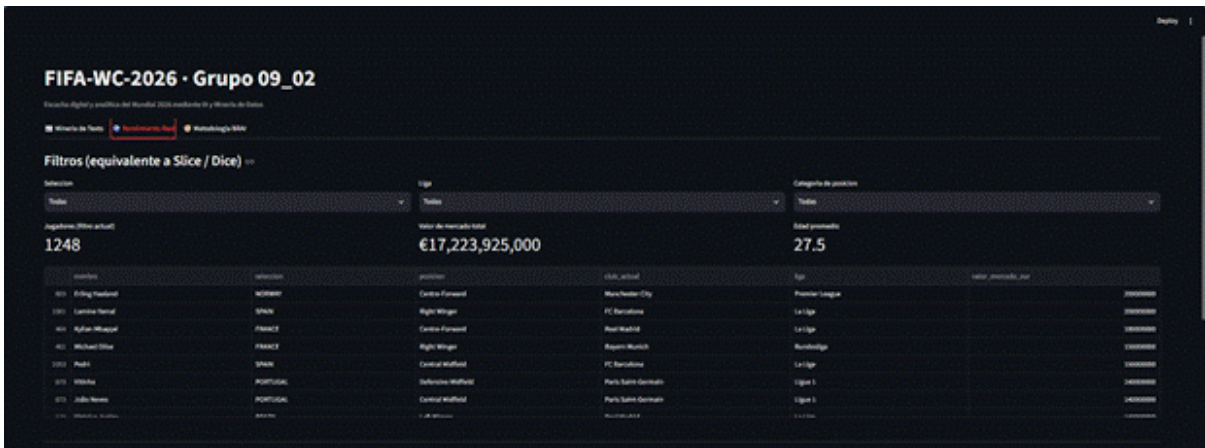


Figura 3. Pestaña Rendimiento Real — filtros interactivos (Slice/Dice) sobre el cubo OLAP de 1,248 jugadores.



Figura 4. Operaciones Roll-up (valor de mercado por grupo) y Pivot (Liga × Categoría de posición).

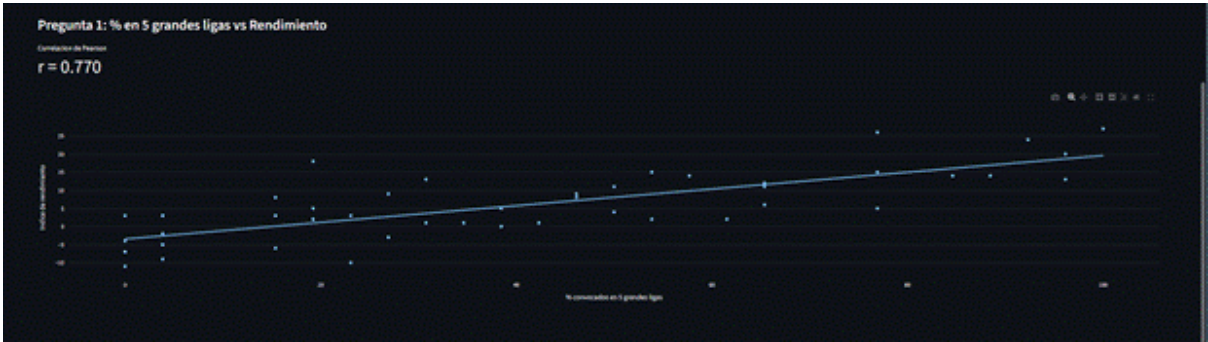


Figura 5. Diagrama de dispersión de la Pregunta 1, con $r = 0.770$.

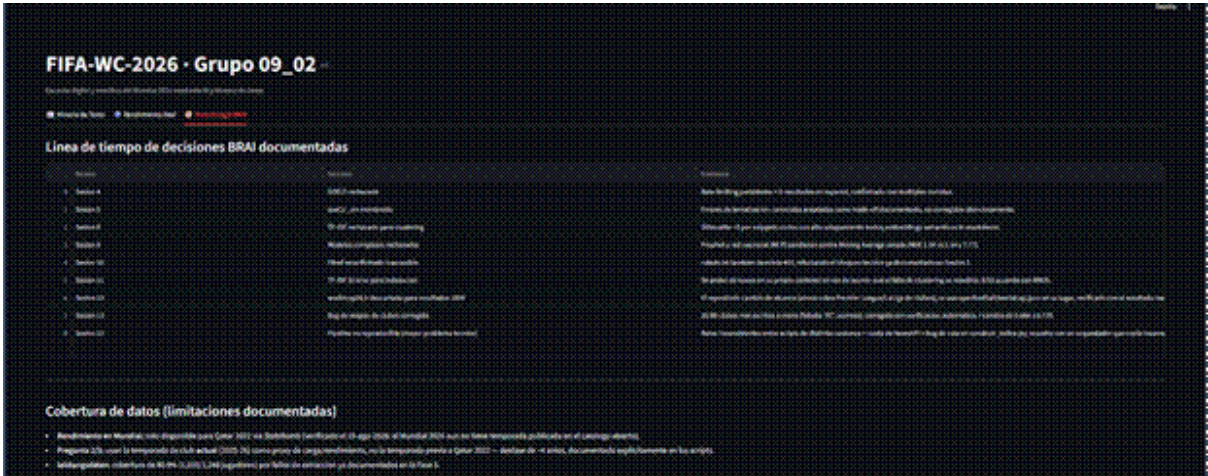


Figura 6. Pestaña Metodología BRAI — línea de tiempo de decisiones documentadas.

Problemas Encontrados y Cómo se Resolvieron

#	Problema	Causa raíz	Solución aplicada
1	El scatter de la Pregunta 1 tronaba al no encontrar la librería statsmodels	trendline="ols" en Plotly Express depende de statsmodels, que no estaba instalado en el entorno del usuario.	Se hizo la línea de tendencia opcional: si statsmodels no está disponible, el gráfico se muestra sin ella y con un aviso, en vez de romper el dashboard completo.
2	AttributeError: 'str' object has no attribute 'get' al cargar el corpus	La estructura real del JSON era {"metadata": {...}, "corpus": [...]}, no una lista plana de artículos como se había asumido inicialmente.	Se solicitó al usuario la estructura exacta de los 3 archivos JSON involucrados y se reescribió la carga de datos acorde a ella, en vez de seguir adivinando.
3	Los artículos del corpus preprocesado no tenían campo 'titulo' para mostrar en resultados de búsqueda	Ese campo solo existe en el corpus crudo original, no en la versión preprocesada por spaCy.	Se construyó un mapa url → título cruzando ambos archivos mediante el campo compartido 'url'.
4	Nombre exacto del campo de lemas para el índice BM25 desconocido	El truncamiento de la salida de diagnóstico ([[:800] caracteres) impidió confirmar el nombre exacto del campo lematizado.	Se escribió una función de tokenización flexible que prueba varios nombres candidatos y cae en 'texto_limpio'.split() como respaldo seguro.
5	El dashboard no abría en el navegador al ejecutarlo	El usuario ejecutó "python app.py" en vez de "streamlit run app.py", por costumbre de los scripts anteriores del proyecto.	Se identificó la causa exacta en el log ("missing ScriptRunContext") y se indicó el comando correcto con el flag -m streamlit run.

Tomas de Decisiones Destacadas

- Se decidió no seguir asumiendo la estructura de los archivos de datos tras el primer fallo real en la máquina del usuario; en su lugar se solicitó un diagnóstico exacto de los 3 archivos JSON antes de escribir una segunda versión del código, evitando ciclos repetidos de prueba y error sobre el mismo problema.
- Se adoptó manejo defensivo de archivos faltantes (advertencias visibles en pantalla) en lugar de que la ausencia de un solo archivo interrumpiera la carga de todo el dashboard, priorizando disponibilidad parcial sobre un fallo total.
- Se priorizó la validación automatizada (Streamlit AppTest) antes de cada entrega del código, lo que permitió detectar un error real (dependencia faltante) sin necesidad de que el usuario lo reportara primero.
- Se mantuvo el buscador semántico como funcionalidad interactiva en vivo (con caché de modelo) en lugar de precalcular resultados estáticos, preservando la interactividad real que motivó la elección de Streamlit sobre una alternativa sin instalación.
- Se diseñaron 3 pestañas temáticas (Minería de Texto, Rendimiento Real, Metodología BRAI) en vez de una sola vista extensa, organizando el dashboard según los dos pipelines integrados en la Sesión 13 más una capa de trazabilidad metodológica.

Comparativo: Decisiones de la IA vs. Decisiones del Humano

Problema	Propuesta / acción de la IA	Decisión del humano	Resultado / reflexión
Selección del stack tecnológico del dashboard	Presentó dos opciones con trade-offs explícitos: Streamlit local (interactivo, requiere instalación) vs. HTML/Plotly como artifact (sin instalación, menos interactivo).	Eligió Streamlit local.	Habilitó el buscador semántico en vivo con datos reales, a costa de requerir instalación de dependencias en la máquina del usuario.
Estructura real de los archivos JSON del corpus	Primera versión: asumió una estructura razonable por convención de nombres, documentando el riesgo. Tras el fallo real, cambió de estrategia y solicitó diagnóstico exacto en vez de seguir adivinando campo por campo.	Ejecutó los diagnósticos solicitados en su propia máquina y compartió los resultados exactos.	Corrección certera en un solo intento adicional, en vez de múltiples ciclos de prueba y error sobre la misma estructura.
Validación de que el dashboard funciona correctamente	Ejecutó pruebas automatizadas (Streamlit AppTest) en un entorno sin los datos reales completos del usuario.	Ejecutó el dashboard en su propia máquina con los datos reales del proyecto y confirmó visualmente cada una de las 3 pestañas mediante capturas de pantalla.	Ninguna validación sustituyó a la otra: la del entorno de pruebas detectó errores de código; la del usuario confirmó comportamiento correcto con datos reales.
Manejo de archivos de datos faltantes o incompletos	Decidió usar advertencias no bloqueantes (st.warning) en vez de interrumpir el dashboard completo ante un archivo ausente.	No objetó la decisión; la aceptó por defecto al no solicitar un comportamiento distinto.	El dashboard permanece parcialmente utilizable incluso cuando faltan archivos derivados de etapas aún bloqueadas del pipeline (ej. cuota de NewsAPI agotada).